

Instruction Formats:

R-Type	Opcode (4)	RS (4)	RT (4)	RD (4)	Shift Amount (5)	Funct Field (5)
I-Type	Opcode (4)	RS (4)	RT (4)	Immediate (14)		
J-Type	Opcode (4)	Jump Address (22)				
J-Type (JAL)	Opcode (4)	Empty (4)	J- Register (4)	Empty (14)		
J-Type (JR)	Opcode (4)	J-Register (4)	Empty (18)			

OpCodes:

	OPCODE			
R-Type	0	0	0	0
lw	0	0	1	1
sw	1	0	1	1
beq	0	1	0	0
addi	1	0	0	0
j	0	0	1	0
jal	0	1	1	0
jr	0	1	1	1

Function Fields:

Operation	Funct-Field
AND	00000
OR	00001
ADD	00010
SUB	00011
SLT (set less than)	00100
XOR	00101
XNOR	00110
NOR	00111
NAND	01000
Pass A	01001
Pass B	01010
NOT A	01011
NOT B	01100
SGT (set greater than)	01101
SLE	01110
SGE	01111
SLL	10000
SLR	10001
MUL.LOW	10010
MUL.UP	10011
DIV	10100
MOD	10101
ZERO	...

How to Convert Instructions:

1. Create an instruction:

This processor uses MIPS Assembly instructions. As an example we will use “*addi \$1, \$0, 43*” instruction.

2. Convert assembly code binary:

To be able to convert instructions we first need to understand which type of instruction it is and convert it into binary. In this case *addi* is an I-type instruction.

1) Look-up instructions type and it's format

addi \$1, \$0, 43 == I-Type

Opcode (4) | RS (4) | RT (4) | Immediate Data (14)

2) Convert the instruction to binary

Opcode (4) | RS (4) | RT (4) | Immediate Data (14)

1000 | 0000 | 0001 | 00000000101011

3. Convert binary to hexadecimal:

To be able to convert our binary value into a hexadecimal value we must add padding in front of our value since hex digits represents 4 bits each. And then divide the binary value into 4 bit parts

1) Add Padding

(00)1000 | 0000 | 0001 | 00000000101011

2) Divide into 4 bit parts

(00)10 | 0000 | 0000 | 0100 | 0000 | 0010 | 1011

3) Convert it into hexadecimal

2 | 0 | 0 | 4 | 0 | 2 | b

Multi-Cycle State Table:

